GReF: A Unified Generative Framework for Efficient Reranking via Ordered Multi-token Prediction

Zhijie Lin*
Kuaishou Technology
Beijing, China
linzhijie@kuaishou.com

Zhuofeng Li*
Shanghai University
Shanghai, China
zhuofengli12345@gmail.com

Chenglei Dai†
Kuaishou Technology
Beijing, China
daichenglei@kuaishou.com

Wentian Bao
Independent
Beijing, China
wb2328@columbia.edu

Shuai Lin
Kuaishou Technology
Beijing, China
shuailin97@gmail.com

Enyun Yu
Independent
Beijing, China
yuenyun@126.com

Haoxiang Zhang
Shanghai University
Shanghai, China
Isaac_GHX@shu.edu.cn

Liang Zhao
Emory University
Atlanta, GA, USA, USA
liang.zhao@emory.edu

arXiv:2510.25220v1 [cs.IR] 29 Oct 2025

Abstract

In a multi-stage recommendation system, reranking plays a crucial role in modeling intra-list correlations among items. A key challenge lies in exploring optimal sequences within the combinatorial space of permutations. Recent research follows a two-stage (generator-evaluator) paradigm, where a generator produces multiple feasible sequences, and an evaluator selects the best one. In practice, the generator is typically implemented as an autoregressive model. However, these two-stage methods face two main challenges. First, the separation of the generator and evaluator hinders end-to-end training. Second, autoregressive generators suffer from inference efficiency. In this work, we propose a Unified Generative Efficient Reranking Framework (GReF) to address the two primary challenges. Specifically, we introduce Gen-Reranker, an autoregressive generator featuring a bidirectional encoder and a dynamic autoregressive decoder to generate causal reranking sequences. Subsequently, we pre-train Gen-Reranker on the item exposure order for high-quality parameter initialization. To eliminate the need for the evaluator while integrating sequence-level evaluation during training for end-to-end optimization, we propose post-training the model through Rerank-DPO. Moreover, for efficient autoregressive inference, we introduce ordered multi-token prediction (OMTP), which trains Gen-Reranker to simultaneously generate multiple future items while preserving their order, ensuring practical deployment in real-time recommender systems. Extensive offline experiments demonstrate that GReF outperforms state-of-the-art reranking methods while achieving latency that is nearly comparable to non-autoregressive models. Additionally, **GReF has also been deployed in a real-world video app Kuaishou with over 300 million daily active users, significantly improving online recommendation quality.**

CCS Concepts

• **Information systems** → **Retrieval models and ranking**.

Keywords

Re-ranking, Recommendation System, Autoregressive Models

ACM Reference Format:
Zhijie Lin, Zhuofeng Li, Chenglei Dai, Wentian Bao, Shuai Lin, Enyun Yu, Haoxiang Zhang, and Liang Zhao. 2025. GReF: A Unified Generative Framework for Efficient Reranking via Ordered Multi-token Prediction. In . ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Multi-stage recommendation systems are widely used on platforms like YouTube and Kuaishou. In the final stage, reranking refines the top candidates by applying specific objectives to further improve recommendation quality. Specifically, it takes the ranking list as input and reorders it by modeling cross-item interactions within the listwise context. Thus, the main challenge in reranking is exploring the optimal sequence within the vast space of permutations.

Extensive research on reranking typically falls into two categories: one-stage and two-stage methods. One-stage methods [4, 30] take candidate items as input, estimate refined scores for each within the permutation, and rerank them greedily based on these scores. However, one-stage methods encounter an inherent contradiction [15, 43]: the reranking operation modifies the permutation, introducing different mutual influences between items compared to the initial arrangement. Consequently, the refined score conditioned on the initial permutation is considered implausible.

*Both authors contributed equally to this research.
†Corresponding Author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

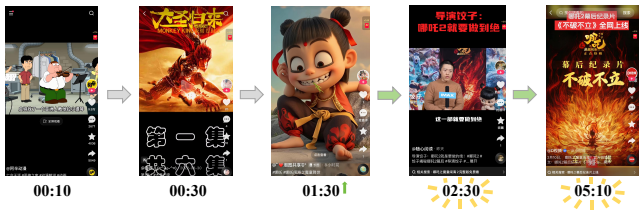


Figure 1: User browsing behavior with causal dependencies

To tackle this challenge, two-stage methods [14, 24, 35, 36] adopt a generator-evaluator framework, where the generator produces multiple candidate sequences and the evaluator selects the optimal one based on an estimated listwise score. Compared to one-stage methods that score items individually, two-stage approaches refine the entire sequence using a sequence-level evaluator, better capturing contextual dependencies and aligning with users' overall preferences.

In practice, the generator is typically implemented as an autoregressive model, such as Seq2Slate [7], to capture the causal dependencies that are widely present in user browsing behavior. As illustrated in Figure 1, a user who watches *Monkey King: Hero is Back* may subsequently browse a related Chinese animated film like *Nezha*, followed by additional content from the Nezha series, eventually spending several minutes on the final video. Such widely observed causal browsing patterns motivate the use of autoregressive models, which are well-suited to capturing the temporal and logical progression of user interactions. .

Challenges. While promising, these two-stage approaches face two significant challenges in real-time industrial recommendation systems. (1) *The separation of the generator and evaluator severely hinders end-to-end training.* After the generator produces potential sequences, conventional two-stage approaches rely on an additional evaluator to select those that best align with user preferences. However, the separation impedes end-to-end training, resulting in a sharp increase in system complexity, misaligned optimization objectives, and limited generalization. (2) *Autoregressive generators suffer from inference efficiency.* Autoregressive models adopt a sequential approach to generate target sequences item by item, resulting in slow inference as the time complexity increases linearly with the sequence length.

Our Work. To address the challenges mentioned above, we propose a Unified **G**enerative[1] Efficient **Re**ranking **F**ramework, named GReF. Specifically, we propose Gen-Reranker, an autoregressive model with a bidirectional encoder and a decoder that generates causal recommendation sequences through dynamic matching. Then we pre-train Gen-Reranker on the large-scale unlabeled item exposure order[2] from the recommendation system to efficiently initialize the model parameters. We then develop specialized strategies to address key challenges of the separation of the generator and evaluator and autoregressive inference efficiency: **First, to unify sequence-level evaluation during training for end-to-end optimization** (**challenge 1**), we propose post-training Gen-Reranker through *Rerank-DPO*, which constructs pair-wise objectives to compare user-preferred recommendation sequences with less-preferred ones, thereby directly integrating sequence-level user preferences during training without the need for an additional evaluator. **Second, to enable efficient autoregressive inference in real-time industrial scenarios** (**challenge 2**), we design *Ordered Multi-token Prediction (OMTP)*, which trains Gen-Reranker to simultaneously generate multiple future items while preserving their order, thereby substantially reducing the latency of autoregressive inference in real-world recommendation systems.

To summarize, our contributions are listed as follows:

- We propose an end-to-end generative reranking framework that integrates the Gen-Reranker model into a unified training pipeline. This design effectively unifies traditional two-stage methods by bridging the gap between the generator and the evaluator, enabling end-to-end optimization.
- We introduce OMTP, a novel method that simultaneously generates multiple future items while preserving their order. OMTP substantially improves the efficiency of autoregressive generation and facilitates seamless deployment of GReF in real-time recommendation systems.
- We conduct extensive offline experiments on both a public dataset and a real-world industrial dataset from Kuaishou, demonstrating that GReF outperforms state-of-the-art reranking methods while maintaining latency nearly comparable to non-autoregressive models. **Notably, GReF has been deployed at Kuaishou, delivering significant improvements across multiple metrics.**

2 RELATED WORK

2.1 Reranking in Recommendation Systems

Reranking in recommendation aims to reorder an input item list by modeling inter-item correlations to maximize user feedback. Reranking models typically take the entire list (listwise context) as input and output a reordered sequence. Existing methods can be broadly divided into one-stage and two-stage approaches.

One-stage methods [4, 7, 29, 30, 43] treat reranking as a retrieval task, where the top-k items are greedily selected based on scores from a context-aware model. For example, PRM [30] and DLCM [4] use Transformers [41] or RNNs [11] to model item dependencies and predict click probabilities. However, modifying the item order alters inter-item interactions, making the original scores less reliable.

Two-stage methods [14, 15, 24, 35, 36, 43] adopt a generator-evaluator framework: the generator produces candidate sequences, and the evaluator selects the best one. Generators are typically based on heuristics (e.g., beam search or item swapping) [14, 36, 43], or generative models [7, 15, 18, 24, 35, 47]. PRS [14] uses beam search to generate candidates and scores them with a permutation-wise model. PIER [36] filters top-K candidates using permutation-level interest and employs attention modules for listwise CTR prediction. DCDR [24] introduces a discrete diffusion model using step-wise operations (e.g., swaps), conditioned on expected user responses. NAR4Rec [35] adopts a non-autoregressive model [19] with unlikelihood training and contrastive decoding to enhance intra-list correlation. Building on this, NLGR incorporates neighbor

[1]In this paper, we primarily use the term "generative" to refer to autoregressive GPT-style models [2, 27, 45], which are characterized by causal attention, next-token prediction.

[2]The term "item exposure order" refers to the item sequence ultimately displayed to the user by the recommendation system.

lists to improve training. Despite their effectiveness, these two-stage methods face challenges in end-to-end optimization due to the separation between generation and evaluation.

2.2 Autoregressive Modeling

Autoregressive language models [3, 5, 6, 9, 10, 13, 22, 23, 27, 31, 32, 38–40] have laid a promising foundation for general-purpose AI. At their core lies the autoregressive generation paradigm—predicting the next token based on previous context—a simple yet powerful approach that scales effectively [20] and demonstrates strong generalization and memorization abilities [3].

Autoregressive generation is also gaining popularity in reranking tasks [7, 16, 44, 46]. Seq2Slate [7] employs a seq2seq model [37] to select items step by step. More recently, LLM4PR [44] and LLM4Rerank [16] leverage zero-shot LLMs with Chain-of-Thought prompting for reranking. While effective, their high inference cost makes them impractical for real-time recommendation.

To address these issues, we propose a novel autoregressive reranking generator equipped with an efficient training framework. Our method effectively models item-level causality and user intent while remaining suitable for real-time recommendation scenarios.

3 PRELIMINARY

3.1 Reranking Task

In multi-stage real-time recommendation systems, reranking plays a crucial role in generating the ultimate list of recommended items [26]. Given a set of candidates $X = \{x_1, x_2, \ldots, x_m\}$ from the preceding stage, the goal of reranking is to produce an ordered sequence of recommendations $Y = \{y_1, y_2, \ldots, y_n\}$ that maximizes the likelihood of favorable feedback from user u. Here, m denotes the number of candidates, and n represents the length of the recommended sequence. Typically, m is much larger than n, with m ranging from several tens to hundreds, while n is usually less than 10.

3.2 Autoregressive Reranking

Given a set of candidate items denoted as $X = \{x_1, x_2, \ldots, x_m\}$, autoregressive reranking models factorize the joint probabilities over possible generated sequences $Y = \{y_0, y_1, \ldots, y_{n+1}\}$ into the product of conditional probabilities:

$$p(Y|X;\theta) = \prod_{t=1}^{n+1} p(y_t|y_{0:t-1}, x_{1:m};\theta), \tag{1}$$

where n represents the length of the recommended sequence. The special tokens y_0 (e.g., `[BOS]`) and y_{n+1} (e.g., `[EOS]`) signify the beginning and end of the target sequence, respectively. In contrast to natural language processing (NLP) tasks, where the sequence length may vary, the sequence length in autoregressive reranking is fixed.

In training, the objective is to optimize the following likelihood using a cross-entropy loss at each timestep:

$$\mathcal{L}_{\text{AR}} = -\log p_{\text{AR}}(Y|X;\theta) = -\sum_{t=1}^{n+1} \log p(y_t|y_{0:t-1}, x_{1:m};\theta). \tag{2}$$

In inference, autoregressive reranking models generate the target sequence sequentially through next-item prediction, effectively modeling the distribution of the target sequence. This property makes them particularly well-suited for reranking tasks, especially given the large combinatorial space of possible permutations.

3.3 Direct Preference Optimization (DPO)

DPO [33] is one of the most popular preference alignment methods in the LLM post-training stage. Rather than explicitly learning a reward model [28], DPO reparameterizes the reward function r by leveraging a closed-form expression with the optimal policy:

$$r(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x), \tag{3}$$

where π_θ is the policy model, π_{ref} is the reference policy, typically the supervised fine-tuned (SFT) model, and $Z(x)$ is the partition function. By incorporating this reward formulation into the Bradley-Terry (BT) ranking objective [8], $p(y_w \succ y_l \mid x) = \sigma\left(r(x, y_w) - r(x, y_l)\right)$, DPO represents the probability of preference data with the policy model, yielding the following objective:

$$-\min_{\pi_\theta} \mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right], \tag{4}$$

where π_θ is the policy model, π_{ref} is the reference policy, and (x, y_w, y_l) represents preference pairs consisting of the prompt x, the winning response y_w, and the losing response y_l from the preference dataset $\mathcal{D}$.

4 GENERATIVE RERANKING FRAMEWORK

In this section, we provide a detailed introduction to our unified generative framework for effective reranking illustrated in Figure 2. We first describe the structure of our Gen-Reranker model in Section 4.1, which generates the causal reranking sequence in an autoregressive manner. We then explore the training mechanism of Gen-Reranker. As detailed in Section 4.2, we first pre-train Gen-Reranker to achieve high-quality parameter initialization and enhance generalization. Building on this, Section 4.3 introduces *Rerank-DPO*, which further aligns the model with user preferences and improves personalization. Finally, to enable efficient autoregressive generation, we introduce Ordered Multi-Token Prediction (OMTP), which generates items simultaneously in a structured order in Section 4.4.

4.1 Gen-Reranker

Autoregressive reranking generates recommendations item by item, capturing complex causal dependencies for more personalized and accurate recommendations. While promising, the effectiveness of autoregressive reranking is hindered by the billion-scale and evolving item vocabularies [3] in recommendation systems, in contrast to the relatively static 100K-scale vocabularies in language models [9]. To address this, we propose *Gen-Reranker*, which comprises two key components: an encoder that leverages bidirectional attention to extract embeddings for each candidate in current session, and a decoder that autoregressively generates the next-item representation and then dynamically matches it with the candidate embeddings

[3] In autoregressive reranking, an item is equivalent to a token in NLP.

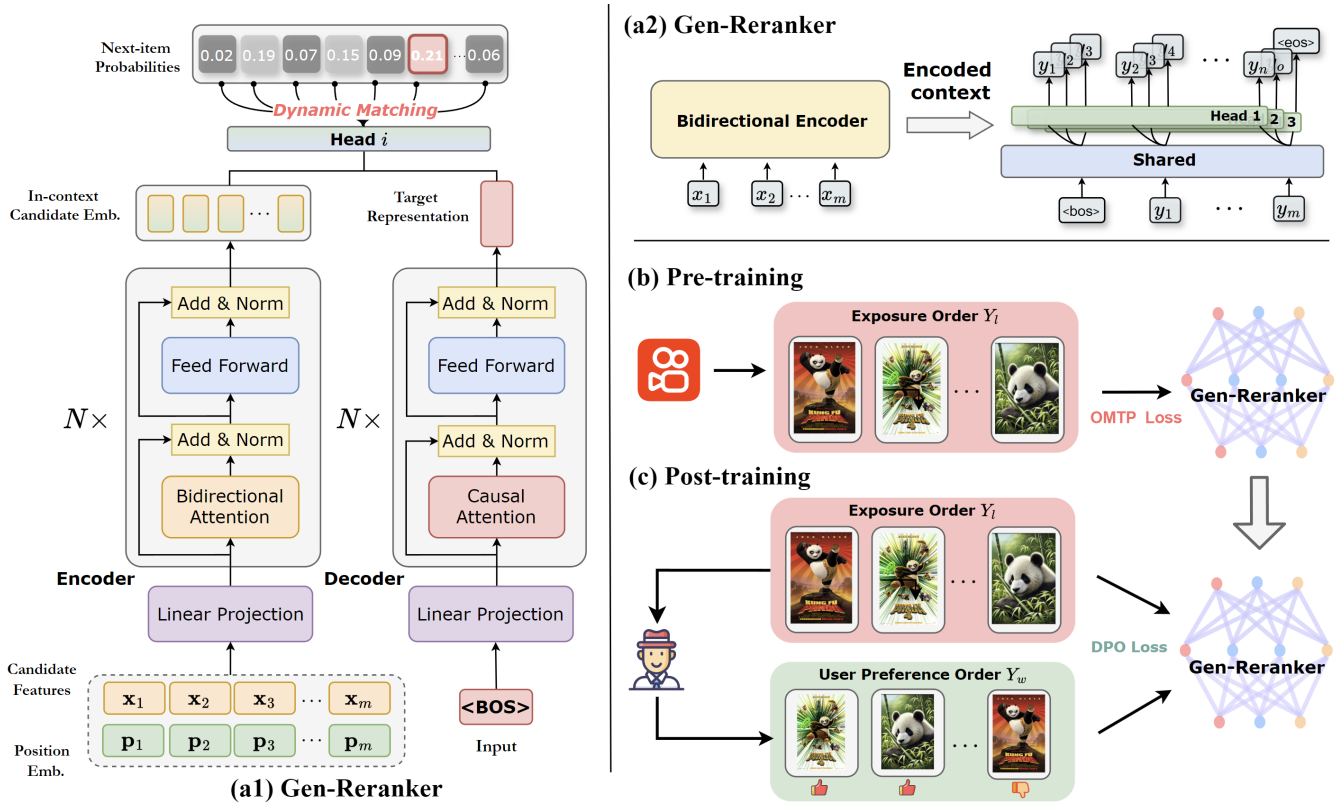


Figure 2: An overview of GReF. GReF consists of Gen-Reranker, which integrates a bidirectional encoder and a dynamic autoregressive decoder with dynamic matching (a1). Specifically, (a2) provides a detailed explanation of how Gen-Reranker generates multiple future items simultaneously through OMTP. Additionally, GReF employs an effective training framework that combines pre-training (b) and post-training (c).

to predict the next item, thereby eliminating the need to consider billions of candidates across the entire recommendation system.

Candidate Items Embeddings. We employ a bidirectional transformer [41] as encoder to generate target-aware embeddings for the candidate items.

Considering a candidate item set $X = \{x_1, x_2, \ldots, x_m\}$, each item x_i is represented by item feature $\mathbf{x}_i \in \mathbb{R}^d$. These features are stacked into a matrix $\mathbf{X} \in \mathbb{R}^{m\times d}$. Additionally, a position embedding $\mathbf{p}_i \in \mathbb{R}^d$ is randomly initialized for each position i, forming a matrix $\mathbf{P} \in \mathbb{R}^{m\times d}$. Note that this position order comes from the previous ranking stage. The input representations are then obtained by summing the item features $\mathbf{X}$ and their corresponding position embeddings $\mathbf{P}$. Subsequently, we input them into the bidirectional encoder. Finally, the final hidden state $\mathbf{Z} = \{\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_m\}$ serves as the candidate embeddings, seamlessly integrating in-context item information.

Dynamic Multi-Item Prediction. To achieve high efficiency in real-time autoregressive generation, we propose *dynamic multi-item prediction*. We replace the standard output layer over the full vocabulary with a concatenation of embeddings corresponding to the candidate items to be reranked. This allows the model to generate outputs solely from the in-context candidate set, eliminating the need to compute over the entire billion-scale item pool in recommendation systems. Moreover, instead of generating one item at a time, we adopt multi-item prediction to enable parallel decoding of multiple future items, significantly improving inference efficiency.

To be more specific, we replace the output layer's weights with candidate embeddings $\mathbf{Z}$ from encoder to create dynamic in-context item vocabularies. During decoding, we compute similarity scores between the decoder's last hidden state $\mathbf{h}$ and the linear layer weight $\mathbf{Z}$ to obtain logits and then apply the softmax function to compute the next-item probabilities. The process is formulated as follows:

$$p_\theta(y_t|y_{0:t-1}) = \frac{\exp\left(\mathbf{h}_t^\top \mathbf{z}_{y_t}\right)}{\sum_{i=0}^{m} \exp\left(\mathbf{h}_t^\top \mathbf{z}_i\right)}, \tag{5}$$

where $\mathbf{h}_t$ and $\mathbf{z}_{y_t}$ represent the final hidden state and item embedding for target item y_t, respectively.

4.2 Pre-training on Recommender World Knowledge

To train reranking models, traditional approaches [14, 24, 30, 36] typically rely on data with user feedback, such as clicks or likes. While promising, user feedback in real-world recommendation

systems is often extremely sparse [25]. Training solely on these samples can lead to overfitting and hinder generalization.

To address this, similar to how LLMs are pre-trained on vast unlabeled data from the internet, we propose pre-training the model on the item exposure order from the recommendation system. This approach is motivated by two key reasons. First, training data is sufficient. Using item exposure order allows the training samples to include all item data, rather than just the items that have been interacted with by users. Second, learning from the item exposure order provides the model with high-quality parameter initialization. A well-trained multi-stage recommendation system, such as YouTube or Kuaishou, incorporates broader world knowledge beyond just user interactions, including expert rules and contextual information. This means that pre-training on the item exposure order generated from modern recommendation systems not only allows the AR model to accurately and comprehensively capture users' interests but also enhances its generalization and robustness.

Specifically, given multiple candidate item sets $\mathcal{X}_{train} = \{X_i | X_i = \{x_1, x_2, \ldots, x_m\}\}_{i=1}^{K}$, We can obtain the ordered exposure sequences $\mathcal{Y}_{train} = \{Y_i | Y_i = \{y_1, y_2, \ldots, y_n\}\}_{i=1}^{K}$ presented to the user from the recommendation system, where $\mathcal{Y}_{train}$ is a subset of $\mathcal{X}_{train}$. Subsequently, we add a special `[BOS]` token at the beginning and a `[EOS]` token at the end of each Y_i to obtain target sequences $\mathcal{Y}_{train} = \{Y_i \mid Y_i = \{y_0, y_1, \ldots, y_n, y_{n+1}\}\}_{i=1}^{K}$. Finally, we pre-train our autoregressive reranking model on $\mathcal{Y}_{train}$ through cross-entropy loss as described in Section 3.2:

$$\mathcal{L}_{pre-train} = -\frac{1}{K} \sum_{\mathcal{Y}_{train}} \sum_{t=1}^{n+1} \log p_\theta(y_t | y_0, y_1, \cdots, y_{t-1}), \tag{6}$$

where p_θ denotes an item distribution predictor with a model parameterized by θ.

4.3 Post-training on User Preferences

A key remaining challenge is how to further integrate user preferences into the model. Conventional two-stage approaches address this by using the evaluator to select the sequences generated from the generator that best align with user preferences. However, this pattern hinders end-to-end training, leading to significantly increased system complexity, misaligned optimization objectives, and limited generalization.

To address this, inspired by DPO [34] in LLMs (with more details provided in Section 3.3), we propose *Rerank-DPO*, which first constructs preference pairs based on item exposure order and user feedback, and then forms a pairwise optimization objective from these pairs, directly integrating sequence-level user preferences into the model.

Specifically, given an exposure order $Y = \{y_1, y_2, \ldots, y_n\}$, we first assess an item's personalization score based on two criteria: (1) original exposure position and (2) user feedback. Utilizing these, our objective is to construct winning and losing sequences that capture both user and recommendation system preferences, ensuring that the integration of user preferences does not interfere with the knowledge acquired during the pre-training stage. Thus we can define the personalization score of item y_i as:

$$S_i = \alpha * \frac{1}{P_i} + \gamma * U_i, \tag{7}$$

where P_i represents the position of y_i in the exposure order, and U_i denotes user feedback on y_i, such as clicks, with values of 0 or 1. α and γ denote the balance hyper-parameter. Afterward, we can rank items in the exposure order Y according to the score to get the personalized sequence Y_w. For example, if the user clicks on item y_2 and y_5 in Y, we can derive a personalized sequence $Y_w = \{y_2, y_5, y_1, y_3, y_4, \ldots, y_n\}$. Finally, if Y_w differs from the original exposure order, we treat Y_w as the winning sequence and exposure order, denoted as Y_l, as the losing sequence, respectively.

Based on this, we can construct the preference dataset $\mathcal{Y}_{post-train} = \{(Y_w, Y_l)_i\}_{i=1}^{K}$ and post-train our pre-trained Gen-Reranker through DPO to align with user preferences. The loss function is defined as follows:

$$\mathcal{L}_{dpo} = \tag{8}$$
$$-\min_{\pi_\theta} \mathbb{E}_{(Y_w, Y_l) \sim \mathcal{Y}_{post-train}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(Y_w)}{\pi_{\text{ref}}(Y_w)} - \beta \log \frac{\pi_\theta(Y_l)}{\pi_{\text{ref}}(Y_l)} \right) \right],$$

where π_θ is the pre-trained Gen-Reranker, π_{ref} is the frozen version, and $\pi_\theta(Y_w)$, $\pi_{\text{ref}}(Y_w)$, $\pi_\theta(Y_l)$, and $\pi_{\text{ref}}(Y_l)$ represent the summed log probabilities of sequences Y_w and Y_l on π_θ and π_{ref}, respectively. β is the hyper-parameter.

4.4 Ordered Multi-token Prediction

To achieve high inference efficiency in real-time autoregressive generation for real-world applications, we propose Ordered Multi-token Prediction (OMTP), which trains Gen-Reranker with n output heads to generate multiple future items in a single forward pass while preserving their order.

Specifically, given the observed context $y_{0:t-1}$, we first encode it into the latent representation $h_{0:t-1}$ via the shared trunk Gen-Reranker, which is then fed into the n output heads to generate predictions for the future items. This design leads to the following cross-entropy loss [17]:

$$\begin{aligned} \mathcal{L}_n &= -\sum_t \log p_\theta(y_{t:t+n-1} \mid h_{0:t-1}) \cdot p_\theta(h_{0:t-1} \mid y_{0:t-1}) \\ &= -\sum_t \sum_{i=0}^{n-1} \log p_\theta(y_{t+i} \mid h_{0:t-1}) \cdot p_\theta(h_{0:t-1} \mid y_{0:t-1}), \end{aligned} \tag{9}$$

where $p_\theta(y_{t+i} \mid y_{0:t-1})$ from head i is computed through Eq. 5.

To further ensure that the multiple items generated by different heads follow a sequential order, which is crucial for modeling user behavior in short-video recommendation, we construct positive and negative item pairs and apply a pairwise loss to capture their relative order. Specifically, at step t, we enumerate permutations of the outputs from n heads. Suppose a permutation Y_+ receives a higher score, measured by a scoring function S such as NDCG based on user clicks. In that case, we treat it as a positive example and another as negative Y_-, thereby guiding the model to learn the correct item sequence:

$$\mathcal{L}_o = -\sum_{t, S(Y_+) > S(Y_-)} \log \sigma(P_\theta(Y_+ \mid y_{0:t-1}) - P_\theta(Y_- \mid y_{0:t-1})). \tag{10}$$

The OMTP loss is defined as:

$$\mathcal{L}_{omtp} = \lambda_1 * \mathcal{L}_n + \lambda_2 * \mathcal{L}_o, \tag{11}$$

where λ_1 and λ_2 are hyper-parameters.

4.5 Optimization and Inference

In the pre-training stage, we train Gen-Reranker on large-scale unlabeled item exposure sequences optimized via the loss function $\mathcal{L}$omtp. Subsequently, we post-train Gen-Reranker using Rerank-DPO, supervised by the loss $\mathcal{L}dpo$. During inference, OMTP predicts multiple items in parallel. To avoid duplicates, we apply a binary mask at each step to exclude previously selected items, enabling efficient autoregressive decoding for real-time industrial applications.

5 EXPERIMENTS

In this section, we present extensive offline experiments and online A/B tests to validate the effectiveness of GReF. We first introduce the baselines and implementation details in Section 5.1. In Section 5.2, we compare GReF with existing baselines in terms of performance and inference time. We also conduct ablation studies to assess the contributions of different training stages and OMTP. To further demonstrate the effectiveness of GReF in real-time recommendation systems, we perform online A/B tests to evaluate our proposed methods in Section 5.3.

5.1 Experiment settings

5.1.1 Datasets. To validate the effectiveness of our framework, we conduct extensive experiments on both public and industrial datasets:

- **Avito**[4]: A public dataset of user search logs from avito.ru, containing over 53 million lists, 1.3 million users, and 36 million ads. Each sample is a search page with multiple ads. Logs from the first 21 days are used for training and the last 7 days for testing. Each sequence has 5 ads. The task is to predict item-wise click-through rates given list-wise inputs.
- **Kuaishou**: Collected from the Kuaishou short-video app with over 300 million daily active users. Each sample is a user request containing user features, 30 candidate items, and 10 exposed items. The dataset includes 300 million users, 733 million items, and 252 million requests. The task for the Kuaishou dataset is to predict whether an item is chosen to be one of the exposed 10 items.

5.1.2 Baselines. We compare our model with 7 state-of-the-art reranking methods. We select DNN and DCN as pointwise baselines, PRM as a one-stage listwise baseline, and Edge-Rerank, PIER, Seq2Slate, and NAR4Rec as two-stage baselines. A brief overview of these methods is provided below:

- **DNN** [12]: DNN a multi-layer perception for click-through rate prediction.
- **DCN** [42]: DCN enhances DNNs with feature crossing at each layer.
- **PRM** [30]: PRM uses self-attention to model item correlations and ranks items by predicted scores.
- **Edge-Rerank** [18]: Edge-Rerank generates a context-aware sequence using adaptive beam search applied to the estimated scores.
- **Seq2Slate** [7]: Seq2Slate leverages pointer networks to select the next item based on previously chosen ones.
- **PIER** [36]: PIER filters top-k candidates via hashing, then jointly trains a generator and evaluator for better permutations.
- **NAR4Rec** [35]: NAR4Rec combines a non-autoregressive generator with unlikelihood training, contrastive decoding, and a reranking evaluator.

5.1.3 Implementation Details. We implement our model in Tensorflow [1] and perform all experiments on four NVIDIA T4 GPU. We configure the Bidirectional Encoder and Dynamic Autoregressive Decoder in our Gen-Reranker with 4 stacked Transformer layers each. For Rerank-DPO, we set the balance hyperparameters to $\alpha = 1$ and $\gamma = 1$, treating clicks as user feedback with values of 0 or 1 while adopting $\beta = 0.1$ to control the scaling of the reward difference. For OMTP, we set $n = 4$ to match the Kuaishou app's UI, which displays four videos per screen. Both α and β are set to 1. During training, we use Adam [21] with an initial learning rate $\eta = 5e^{-6}$ to update the parameters, and the batch size is set to 1,024. We first pre-train Gen-Reranker and then post-train it using Rerank-DPO. We run our model and other baseline methods five times with different random seeds. Additionally, we either optimize the parameters or follow the settings in the original papers to achieve the best performance for the baseline methods.

5.1.4 Metrics. Following previous work [24, 35, 36], we evaluate different methods in offline experiments using AUC and NDCG on both the Avito and Kuaishou datasets.

5.2 Offline experiments

5.2.1 Performance comparison. Here we show the results of our proposed method GReF. As shown in Table 1, GReF outperforms all baseline methods on both the Avito and KuaiShou datasets across key evaluation metrics. On the Avito dataset, GReF achieves the highest AUC, about 1.5% higher than the next best model, and the best NDCG, approximately 0.8% higher than PIER. Similarly, on the KuaiShou dataset, GReF leads with an AUC, about 1.4% higher than the best-performing baseline, and an NDCG, around 0.5% better than other methods. The consistent performance across both datasets highlights GReF's robustness and ability to enhance recommendation quality, making it a highly effective approach in practical scenarios.

5.2.2 Inference Time comparison. We evaluate GReF's efficiency by comparing inference times on the Kuaishou dataset (Table 2) using a Tesla T4 16G GPU, measuring average single-sample latency. Classical models like DNN and DCN are highly efficient due to shallow architectures, while PRM, ERK, and PIER incur moderately higher latency. The non-autoregressive NAR4Rec achieves 12.67 ms. GReF achieves a strong balance between accuracy and efficiency, outperforming all baselines in AUC and NDCG with an inference time of 12.97 ms, close to NAR4Rec and over 4× faster than Seq2Slate due to ordered multi-token prediction (OMTP). Without OMTP, GReF's step-by-step decoding results in the highest latency. OMTP predicts multiple tokens per step, nearly halving forward passes and latency.

This highlights OMTP's effectiveness in accelerating autoregressive inference, enabling GReF to combine generative expressiveness with real-time deployability.

[4] https://www.kaggle.com/c/avito-context-ad-clicks/data

Table 1: Comparison between GReF and baseline methods on the Avito and KuaiShou datasets. The best results on each dataset are highlighted in bold.

Method	Avito		KuaiShou	
	AUC	NDCG	AUC	NDCG
DNN	0.6614	0.6920	0.6866	0.7122
DCN	0.6623	0.7004	0.6879	0.7116
PRM	0.6881	0.7380	0.7119	0.7396
Edge-rerank	0.6953	0.7203	0.7143	0.7401
PIER	0.7109	0.7401	0.7191	0.7387
Seq2Slate	0.7034	0.7225	0.7165	0.7383
NAR4Rec	0.7234	0.7409	0.7254	0.7425
GReF	**0.7384**	**0.7478**	**0.7387**	**0.7498**

Table 2: Inference time comparison between GReF and baselines on the Kuaishou dataset. Note that ERK denotes Edge-Rerank.

	DNN	DCN	PRM	ERK	PIER	NAR4Rec	Seq2Slate	GReF (w/o OMTP)	GReF
Inf. Time (ms)	8.03	8.25	10.96	10.45	13.69	12.67	67.34	24.29	**12.97**

5.2.3 Ablation Study of Training Stage. To investigate the contributions of different training stages, we conduct the ablation studies and present AUC and NDCG results in Table 3. We can find that (1) Pre-training on exposure order provides the model with high-quality parameter initialization, effectively capturing user interest in many cases. (2) Post-training is essential for further integrating user preferences, leading to an improvement of 0.63% in AUC and 0.45% in NDCG. (3) Relying solely on post-training leads to a significant degradation in model performance. One possible explanation is that directly applying reinforcement learning-based DPO on user feedback data in a cold-start scenario can easily result in training instability and even collapse.

Table 3: Ablation studies of training stages and OMTP loss in the pre-training stage on the Kuaishou dataset.

Training Stages				OMTP Loss			
Pre-training	Post-training	AUC	NDCG	$\mathcal{L}_n$	$\mathcal{L}_o$	AUC	NDCG
✓		0.7361	0.7474			0.7324	0.7453
	✓	0.6832	0.7103	✓		0.7373	0.7484
✓	✓	**0.7387**	**0.7498**	✓	✓	**0.7387**	**0.7498**

5.2.4 Ablation Study of OMTP. To further evaluate the effectiveness of the OMTP objectives $\mathcal{L}_n$ (Eq.9) and $\mathcal{L}_o$ (Eq.10) during pre-training, we conduct ablation studies, with results presented in Table 3. Note that using either loss $\mathcal{L}_n$ and $\mathcal{L}_o$ alone corresponds to single-head autoregressive pre-training as defined in Eq. 6, and all models undergo post-training. We highlight the following findings: (1) The model, which predicts multiple tokens in a single forward pass and is trained using $\mathcal{L}_n$ alone, achieves performance comparable to the full autoregressive framework. This demonstrates that the MTP paradigm can serve as a strong alternative to traditional autoregressive methods. (2) Adding the ordered loss $\mathcal{L}_o$ on top of $\mathcal{L}_n$ leads to further improvements, confirming that explicitly enforcing a preference-consistent generation order significantly enhances model performance. This indicates that OMTP not only greatly accelerates inference compared to autoregressive approaches but also maintains, or even improves, recommendation accuracy.

5.3 Online experiments

To further demonstrate our effectiveness, we conduct online A/B experiments on the Kuaishou app with 8% of the entire production traffic over one week. The online baseline is NAR4Rec [35].

5.3.1 Experimental Results. The experiments have been launched on the system over one week, and the result is listed in Table 4. GReF outperforms NAR4Rec by a large margin. To be more specific, the Views increase by +0.33%, indicating better overall visibility of recommended content. Long Views rise by +0.42% with a p-value less than 0.01 and a Confidence Interval of [0.31%, 0.52%], reflecting enhanced user engagement and prolonged content interaction. Likes show a +1.19% improvement, suggesting higher user satisfaction and content appreciation. Most notably, Forwarding (shares) increases by +2.98%, highlighting the model's effectiveness in encouraging content sharing and expanding its reach. Additionally, Comments increase by +1.78%, indicating that users are more inclined to engage in discussions and express their opinions on the recommended content. This suggests that GReF not only improves content visibility and engagement but also fosters a more interactive and participatory user experience.

Table 4: The experimental results from Online A/B testing. All values are the relative improvements of GReF. For the online A/B test in Kuaishou, the 0.2% increase in views and long view, and 0.5% in user interactions (like, forward, comment) are very significant improvements in our system.

Views	Long Views	Likes	Forwards	Comments
+0.33%	+0.42%	+1.19%	+2.98%	+1.78%

6 Conclusion

In this paper, we provide an overview of the current formulation and challenges associated with reranking in recommendation systems. We emphasize the significance of autoregressive reranking and pioneer the deployment of the autoregressive reranking generator, GenReranker, in real-time recommendation systems. Building on this, we propose GReF, a novel generative reranking framework with end-to-end training. By seamlessly integrating the GenReranker model into an optimized pipeline, GReF not only boosts the effectiveness of autoregressive reranking but also enables efficient real-time deployment. To validate its effectiveness, we conduct extensive offline and online A/B experiments, and the comparison with state-of-the-art reranking methods demonstrates the superiority of our approach. In the future, our future work will focus on further enhance the capabilities of GReF.

7 GenAI Usage Disclosure

The authors affirm that no generative artificial intelligence (GenAI) tools were used in the preparation of this manuscript. All text,

figures, tables, code, and other scholarly contributions were created solely by the authors without the assistance of generative AI software. The authors take full responsibility for the originality, accuracy, and integrity of the content.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.

[2] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).

[3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[4] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *The 41st international ACM SIGIR conference on research & development in information retrieval*. 135–144.

[5] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).

[6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[7] Irwan Bello, Sayali Kulkarni, Sagar Jain, Craig Boutilier, Ed Chi, Elad Eban, Xiyang Luo, Alan Mackey, and Ofer Meshi. 2018. Seq2Slate: Re-ranking and slate optimization with RNNs. *arXiv preprint arXiv:1810.02019* (2018).

[8] Ralph Allan Bradley and Milton E. Terry. 1952. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika* 39 (1952), 324.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *NeurIPS* (2020).

[10] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[11] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555* (2014).

[12] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

[13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[14] Yufei Feng, Yu Gong, Fei Sun, Junfeng Ge, and Wenwu Ou. 2021. Revisit recommender system in the permutation prospective. *arXiv preprint arXiv:2102.12057* (2021).

[15] Yufei Feng, Binbin Hu, Yu Gong, Fei Sun, Qingwen Liu, and Wenwu Ou. 2021. GRN: Generative Rerank Network for Context-wise Recommendation. *arXiv preprint arXiv:2104.00860* (2021).

[16] Jingtong Gao, Bo Chen, Xiangyu Zhao, Weiwen Liu, Xiangyang Li, Yichao Wang, Wanyu Wang, Huifeng Guo, and Ruiming Tang. [n. d.]. LLM4Rerank: LLM-based Auto-Reranking Framework for Recommendations. In *THE WEB CONFERENCE 2025*.

[17] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737* (2024).

[18] Xudong Gong, Qinlin Feng, Yuan Zhang, Jiangling Qin, Weijie Ding, Biao Li, Peng Jiang, and Kun Gai. 2022. Real-time Short Video Recommendation on Mobile Devices. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 3103–3112.

[19] Jiatao Gu, James Bradbury, Caiming Xiong, Victor OK Li, and Richard Socher. 2017. Non-autoregressive neural machine translation. *arXiv preprint arXiv:1711.02281* (2017).

[20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[21] Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *ICLR*.

[22] Shuai Lin, Wentao Wang, Zichao Yang, Xiaodan Liang, Frank F Xu, Eric Xing, and Zhiting Hu. 2019. Data-to-text generation with style imitation. *arXiv preprint arXiv:1901.09501* (2019).

[23] Shuai Lin, Pan Zhou, Xiaodan Liang, Jianheng Tang, Ruihui Zhao, Ziliang Chen, and Liang Lin. 2021. Graph-evolving meta-learning for low-resource medical dialogue generation. In *proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 13362–13370.

[24] Xiao Lin, Xiaokai Chen, Chenyang Wang, Hantao Shu, Linfeng Song, Biao Li, et al. 2023. Discrete Conditional Diffusion for Reranking in Recommendation. *arXiv preprint arXiv:2308.06982* (2023).

[25] Siyi Liu and Yujia Zheng. 2020. Long-tail session-based recommendation. In *Proceedings of the 14th ACM conference on recommender systems*. 509–514.

[26] Weiwen Liu, Yunjia Xi, Jiarui Qin, Fei Sun, Bo Chen, Weinan Zhang, Rui Zhang, and Ruiming Tang. 2022. Neural re-ranking in multi-stage recommender systems: A review. *arXiv preprint arXiv:2202.06602* (2022).

[27] OpenAI. 2023. GPT-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[28] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. 2022. Training language models to follow instructions with human feedback. In *NeurIPS*.

[29] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. Setrank: Learning a permutation-invariant ranking model for information retrieval. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 499–508.

[30] Changhua Pei, Yi Zhang, Yongfeng Zhang, Fei Sun, Xiao Lin, Hanxiao Sun, Jian Wu, Peng Jiang, Junfeng Ge, Wenwu Ou, et al. 2019. Personalized re-ranking for recommendation. In *Proceedings of the 13th ACM conference on recommender systems*. 3–11.

[31] Alec Radford. 2018. Improving language understanding by generative pre-training. (2018).

[32] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning transferable visual models from natural language supervision. In *ICML*.

[33] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *NeurIPS*.

[34] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems* 36 (2024).

[35] Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. 2024. Non-autoregressive generative models for reranking recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5625–5634.

[36] Xiaowen Shi, Fan Yang, Ze Wang, Xiaoxu Wu, Muzhi Guan, Guogang Liao, Wang Yongkang, Xingxing Wang, and Dong Wang. 2023. PIER: Permutation-Level Interest-Based End-to-End Re-ranking Framework in E-commerce. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4823–4831.

[37] I Sutskever. 2014. Sequence to Sequence Learning with Neural Networks. *arXiv preprint arXiv:1409.3215* (2014).

[38] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[39] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[40] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[41] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).

[42] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.

[43] Yunjia Xi, Weiwen Liu, Xinyi Dai, Ruiming Tang, Weinan Zhang, Qing Liu, Xiuqiang He, and Yong Yu. 2021. Context-aware reranking with utility maximization for recommendation. *arXiv preprint arXiv:2110.09059* (2021).

[44] Yang Yan, Yihao Wang, Chi Zhang, Wenyuan Hou, Kang Pan, Xingkai Ren, Zelun Wu, Zhixin Zhai, Enyun Yu, Wenwu Ou, et al. 2024. LLM4PR: Improving Post-Ranking in Search Engine with Large Language Models. *arXiv preprint arXiv:2411.01178* (2024).

[45] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical

report. *arXiv preprint arXiv:2407.10671* (2024).

[46] Haobo Zhang, Qiannan Zhu, and Zhicheng Dou. 2025. Enhancing Reranking for Recommendation with LLMs through User Preference Retrieval. In *Proceedings of the 31st International Conference on Computational Linguistics*. 658–671.

[47] Tao Zhuang, Wenwu Ou, and Zhirong Wang. 2018. Globally optimized mutual influence aware ranking in e-commerce search. *arXiv preprint arXiv:1805.08524* (2018).